End-to-End DevOps Automation Project

End-to-End DevOps Automation using GitHub, Jenkins, Terraform, Ansible, Docker, DockerHub and Docker Swarm

---

1. Project Introduction

This project demonstrates an end-to-end DevOps automation workflow for deploying a web application using GitHub, Jenkins, Docker, DockerHub, Terraform, Ansible and Docker Swarm.

The project starts from the developer side, where the application files are created and pushed to GitHub. Jenkins is then used to automate the application deployment and infrastructure provisioning process.

Terraform is used to create the required AWS infrastructure. After the infrastructure is created, Ansible is used to configure the servers and create the Docker Swarm environment.

Finally, the application stored in DockerHub is deployed on the Docker Swarm cluster and accessed through the public IP address of the worker node.

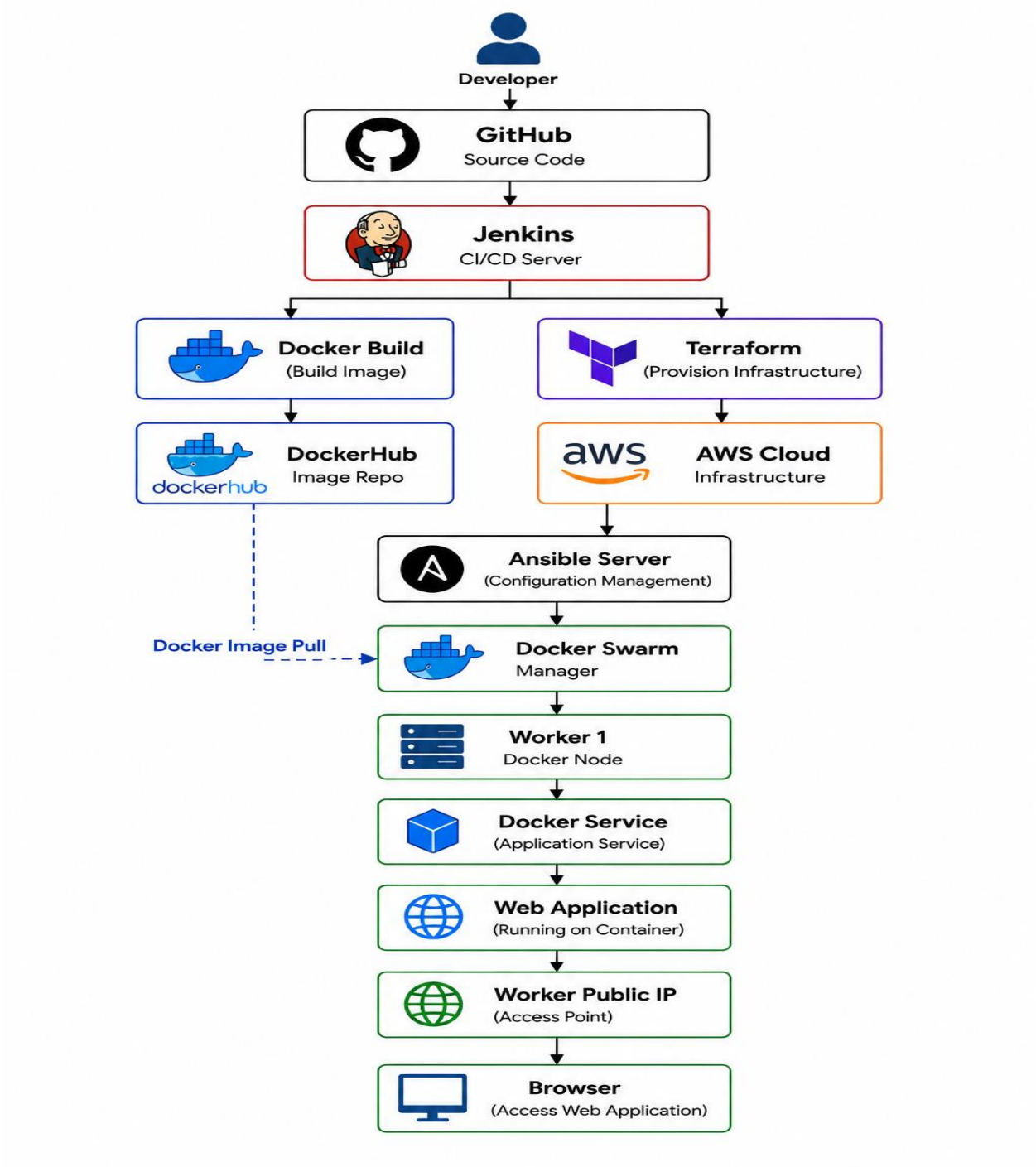
---

2. Technologies Used

Technology	Purpose
Git	Version control
GitHub	Source-code repository
Jenkins	CI/CD automation
AWS	Cloud infrastructure
Terraform	Infrastructure as Code
Ansible	Server configuration and automation
Docker	Application containerization
DockerHub	Container image registry
Docker Swarm	Container orchestration
Linux	Server operating system
Bash/Shell	Automation and server commands
Nginx	Web-server/container base image
HTML	Sample web application

---

3. Project Architecture



4. Project Objective

The main objectives of this project are:

1. Automate application deployment using Jenkins.
2. Store application source code in GitHub.
3. Build a Docker image from the application.
4. Push the Docker image to DockerHub.
5. Provision AWS infrastructure automatically using Terraform.
6. Configure servers automatically using Ansible.
7. Create a Docker Swarm cluster.
8. Deploy the application as a Docker Swarm service.
9. Make the application accessible through a public IP.
10. Demonstrate an end-to-end CI/CD and Infrastructure-as-Code workflow

---

5. Step-by-Step Project Implementation

Step 1: Launch Developer Server

The first step of the project is to launch the developer server.

The developer server is used to create and maintain the application source code and Docker-related files.

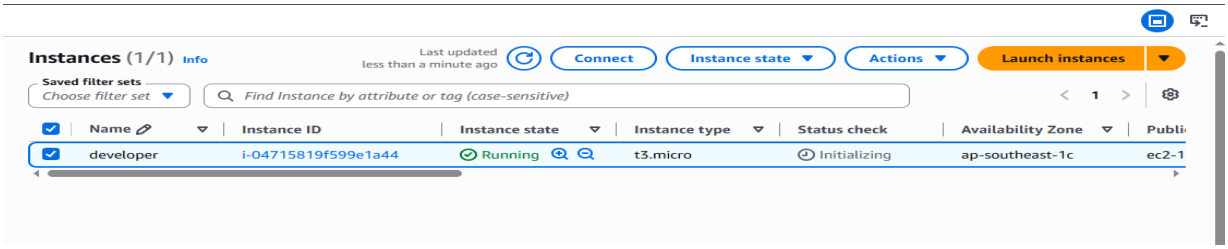
The required application files are created on this server.

Example files:

index.html
Dockerfile

The developer server acts as the starting point of the complete DevOps workflow.

Screenshot:



Step 2: Developer Creates Files and Pushes Them into Git

After launching the developer server, the developer creates the application files.

The web application source code is stored in the application file, while the Dockerfile contains the instructions required to containerize the application. And the also the developer pushes the files such as acn.sh, ansible_lap_setup.tf, node.sh, output.tf, provider.tf, vars.tf.

The files are added to Git using:

git add .

A commit is then created:

git commit -m "Add application files"

The repository is then connected to GitHub.

The basic Git workflow is:

Create Files
    |
    v
git add
    |
    v
git commit
    |
    v
git push

Screenshot:

```
[root@devl punegit]# ls
acn.sh  ansible_lab_setup.tf  node.sh  output.tf  provider.tf  vars.tf
[root@devl punegit]# git  add .
[root@devl punegit]# git commit -m 'myawspro' .
On branch master
nothing to commit, working tree clean
[root@devl punegit]# git status
On branch master
nothing to commit, working tree clean
[root@devl punegit]# git remote add origin https://github.com/diptimokate/devpro
.git
error: remote origin already exists.
[root@devl punegit]# git push https://ghp_hzavSsjxIPxUD6HgWHkoOg0tScVl7Rlk7Ry8@g
ithub.com/diptimokate/devpro.git
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 2 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (14/14), 2.76 KiB | 2.76 MiB/s, done.
Total 14 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/diptimokate/devpro.git
 * [new branch]      master -> master
[root@devl punegit]#
```

Step 3: Files Are Successfully Pushed to GitHub

After committing the files, they are pushed to the GitHub repository.

Example:

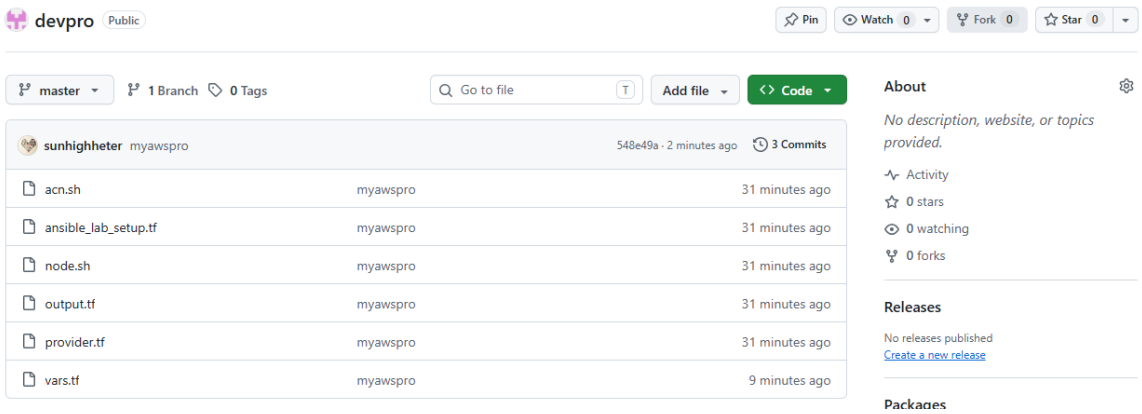
git push -u origin main

The GitHub repository now contains the application source code and Dockerfile.

The repository acts as the centralized source-code repository for the project.

The original documentation confirms that the files were successfully pushed to GitHub.

Screenshot:



Step 4: Launch Jenkins Server

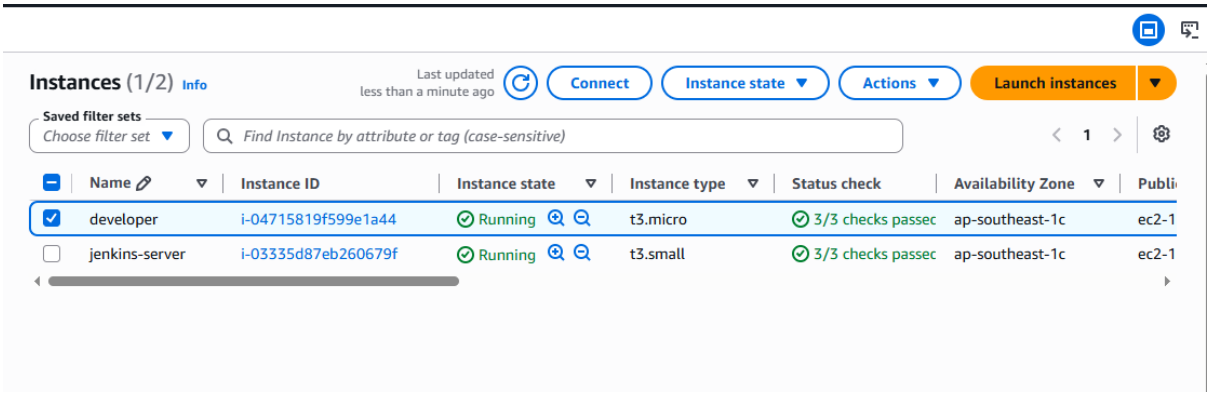
The next step is to launch the Jenkins server.

Jenkins is used as the automation and CI/CD server for the project.

Jenkins performs different tasks throughout the project, including:

- Pulling code from GitHub
- Building Docker images
- Pushing images to DockerHub
- Running Terraform
- Managing infrastructure automation
- Running Ansible-based deployment tasks

Screenshot:



Step 5: Creating the First Jenkins Job – Push Website to DockerHub

The first Jenkins project is created to automate the process of building the application and pushing it to DockerHub.

The basic workflow is:

GitHub
|
Jenkins
|
Application Source Code
|
Docker Build
|
Docker Image
|
DockerHub

Jenkins pulls the application files from GitHub.

The Dockerfile is then used to create the Docker image.

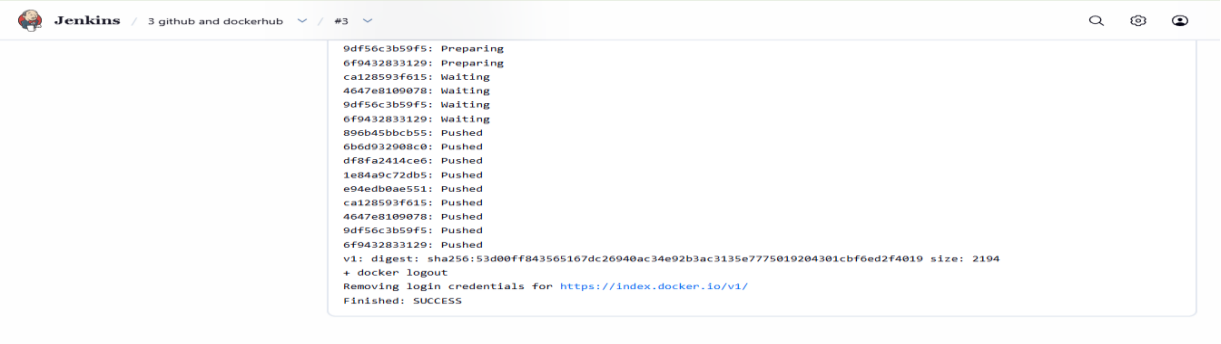
The image is tagged with the DockerHub repository name.

Example:

docker build -t diptimokat/docker_swarm_devops_project:latest .

The image can then be pushed to DockerHub.

Screenshot:



Step 6: Application Source Code Is Pushed to DockerHub

After Jenkins builds the Docker image, the image is pushed to DockerHub.

Example:

docker login

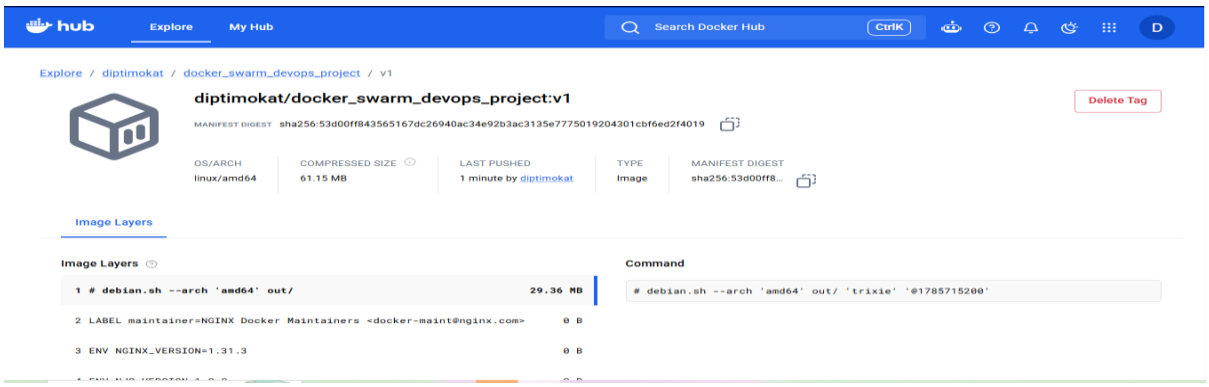
Then:

docker push diptimokat/docker_swarm_devops_project:latest

DockerHub now contains the application image.

The DockerHub repository acts as the image registry from which the application can later be pulled during deployment.

Screenshot:



Step 7: Create the second job in Jenkins for Infrastructure and Upload Pipeline Script

After the application image is available on DockerHub, the infrastructure automation stage begins.

A new Jenkins item is created for infrastructure management.

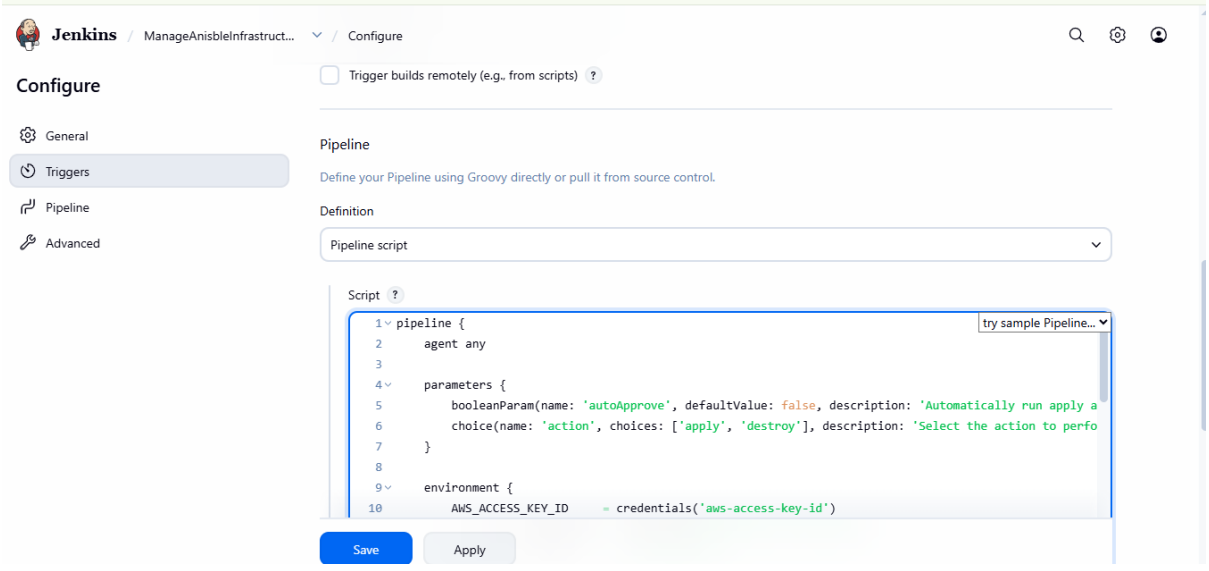
The Jenkins pipeline script is uploaded/configured for the infrastructure provisioning process.

The pipeline is responsible for executing Terraform.

The basic workflow is:

GitHub
  |
  v
Jenkins
  |
  v
Terraform Pipeline
  |
  v
AWS Infrastructure

Screenshot:



Step 8: Jenkins Successfully Runs Terraform Infrastructure

Jenkins executes Terraform to create the required AWS infrastructure.

Terraform is used as Infrastructure as Code (IaC).

Instead of manually creating every server, the infrastructure is defined through Terraform configuration files.

The infrastructure includes:

- Ansible server
- Docker Swarm manager
- Docker Swarm worker

The general Terraform workflow is:

terraform init
terraform validate
terraform plan
terraform apply

Jenkins executes this process automatically.

Screenshot:

Step 9: Launch VPC, Subnet, etc. Through Infrastructure

Terraform also creates the required AWS networking infrastructure.

The networking infrastructure includes resources such as:

- VPC
- Subnet
- Security configuration
- Other required networking resources

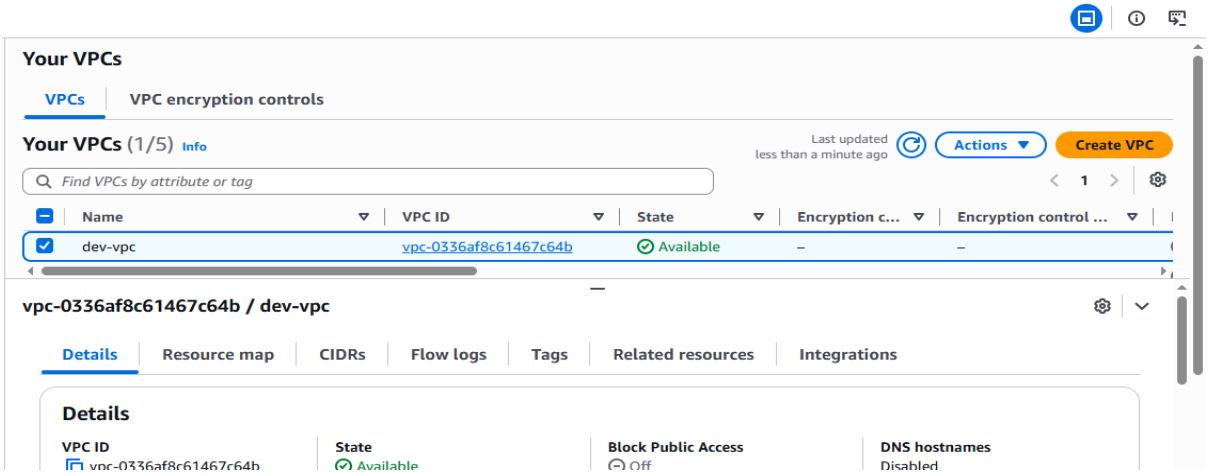
The purpose of this infrastructure is to provide the network environment in which the EC2 instances will operate.

The project documentation specifically records the launch of VPC, subnet and related infrastructure.

The architecture is:

AWS
 |
 +---- VPC
       |
       +---- Subnet
             |
             +---- EC2 Instances

Screenshot:



Step 10: Launch Instances Through Terraform by Jenkins

After creating the required networking infrastructure, Terraform creates the EC2 instances.

The required servers include:

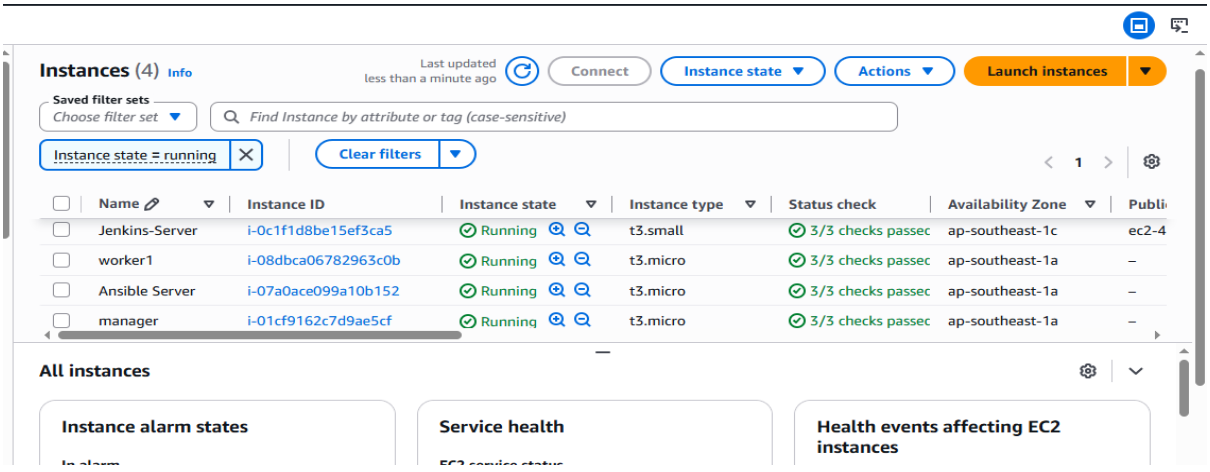
Ansible Server
Manager Node
Worker Node

The infrastructure creation is controlled by Jenkins.

The workflow is:

Jenkins
  |
  v
Terraform
  |
  v
AWS
  |
  +---- Ansible Server
  |
  +---- Manager
  |
  +---- Worker

Screenshot:



Step 11: punepro Directory Available on Ansible Server

After Terraform creates the infrastructure, the Ansible server is accessed.

The project directory named:

punepro

is available on the Ansible server.

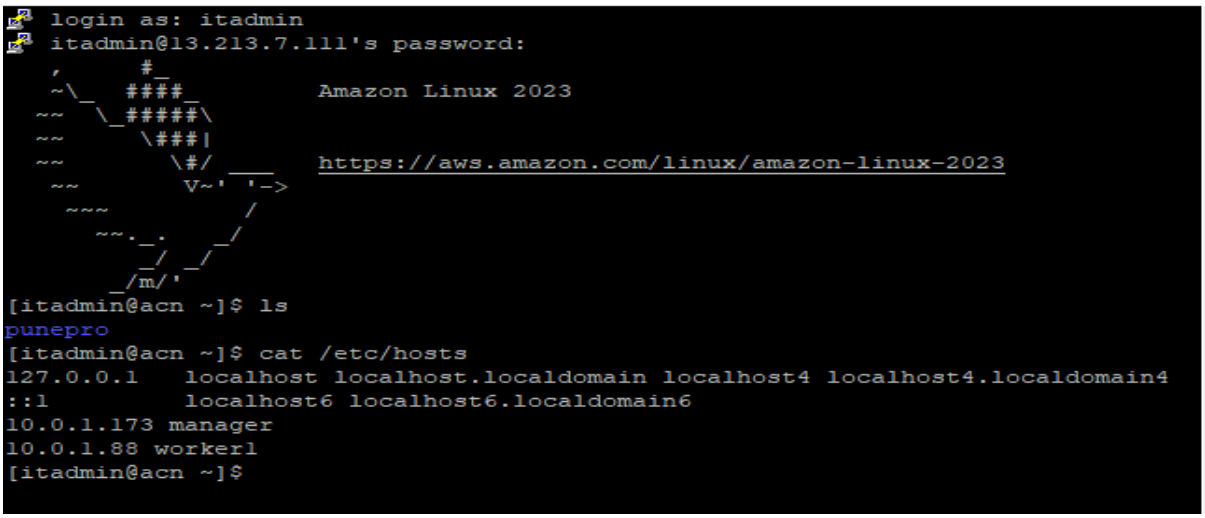
This directory contains the Ansible-related project files used for configuring the Docker environment and Docker Swarm.

Example:

cd /home/itadmin/punepro
ls

The directory contains the required Ansible files/playbooks.

Screenshot:



Step 12 : Creating the Third Job – Docker Swarm Lab Through Jenkins and Ansible

The next stage is to create the Docker Swarm environment.

A Jenkins project is created to execute the Ansible automation.

The workflow is:

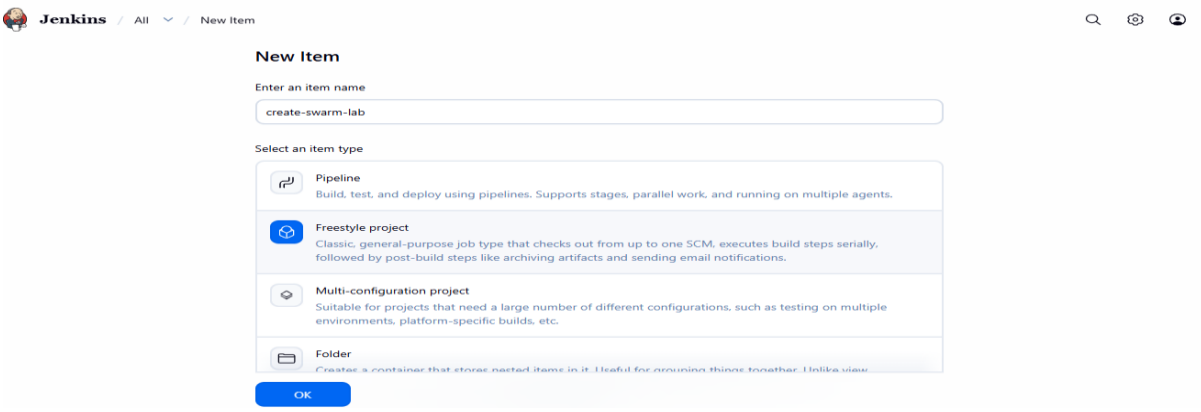
```
Jenkins
  |
  v
Ansible
  |
  +---------------+
  |               |
  v               v
Manager       Worker
  |               |
  +-----+-------+
        |
        v
   Docker Swarm
```

Ansible automates the configuration of the manager and worker nodes.

Typical tasks include:

- Installing Docker
- Starting Docker
- Enabling Docker
- Configuring Docker
- Initializing Docker Swarm
- Joining worker nodes to the Swarm

Screenshot:

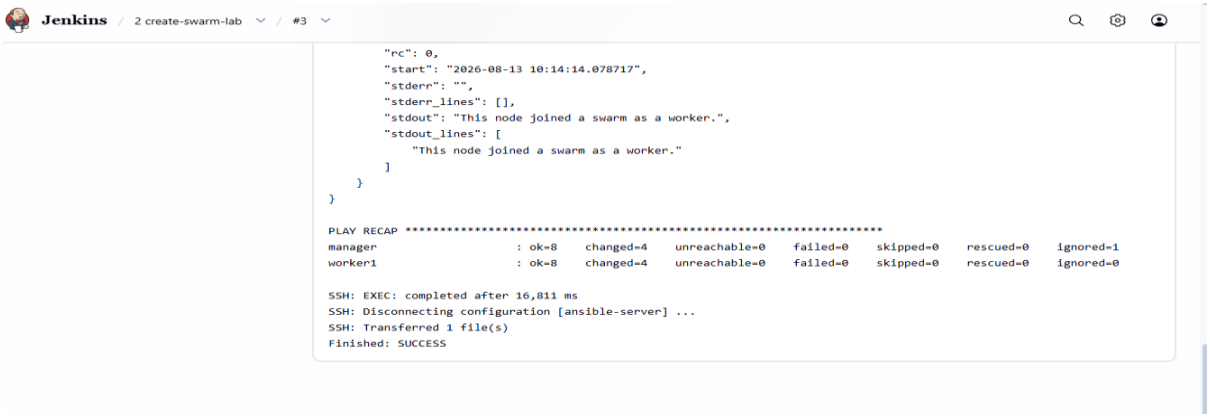


Step 13: Successfully Created Swarm Lab

After Ansible completes successfully, the Docker Swarm cluster is available.

The manager node acts as the control node and the worker node participates in running containers.

Screenshot:



Step 14: Creating the Fourth Job to Deploy the Application Available on DockerHub

After successfully creating the Docker Swarm cluster, the next Jenkins job is created for application deployment.

This project uses the Docker image that was previously pushed to DockerHub.

The deployment flow is:

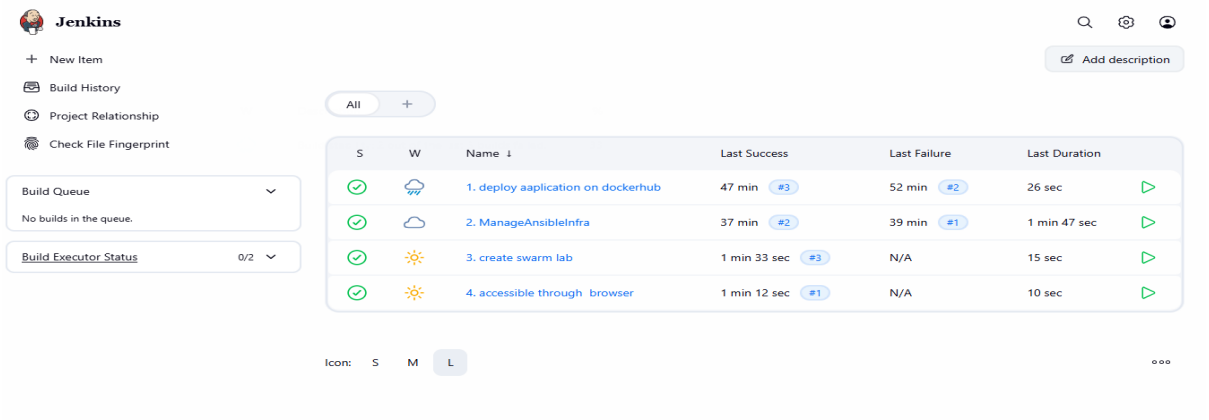
DockerHub
|
v
Application Image
|
v
Ansible / Docker Swarm
|
v
Swarm Service
|
v
Worker Node

The Docker image is pulled from DockerHub and deployed as a Docker Swarm service.

Example:

docker service create --name myweb -p 81:80 --replicas 3 diptimokat/docker_swarm_devops_project:latest

Screenshot:



Step 15: Application Accessible Through Worker Public IP

The final step is to access the deployed application.

Once the Docker Swarm service is successfully deployed, the application is exposed through port 81.

The worker node's public IP is used to access the application from a browser.

Example:

http://<WORKER_PUBLIC_IP>:81

Screenshot:

---

6. Complete Project Flow

The complete project can be summarized as:

STEP 1 : Launch Developer Server
↓
STEP 2 : Create Application Files
↓
STEP 3 : Push Files to GitHub
↓
STEP 4 : Launch Jenkins Server
↓
STEP 5 : Create Jenkins Project for DockerHub
↓
STEP 6 : Push Application Image to DockerHub
↓
STEP 7 : Create Jenkins Infrastructure Item
↓
STEP 8 : Run Terraform Through Jenkins
↓
STEP 9 : Create VPC / Subnet / Infrastructure
↓
STEP 10 : Launch EC2 Instances
↓
STEP 11 : Access punepro Directory on Ansible Server
↓
STEP 12 : Create Swarm Lab Through Jenkins + Ansible
↓
STEP 13 : Verify Docker Swarm
↓
STEP 14 : Create Application Deployment Project
↓
STEP 15 : Access Application Through Worker Public IP

---

7. Final Project Result

The project successfully demonstrates an end-to-end automated DevOps deploymet workflow.

The application starts from the developer server and is pushed to GitHub. Jenkins then automates the application and infrastructure stages. DockerHub stores the application image, while Terraform provisions the AWS infrastructure.

Ansible is used to configure the provisioned servers and create the Docker Swarm environment. The application image stored in DockerHub is then deployed to the Swarm cluster.

Finally, the deployed application becomes accessible through the public IP address of the worker node.

Therefore, the complete project demonstrates the integration of:

Git + GitHub + Jenkins + Docker + DockerHub + Terraform + AWS + Ansible + Docker Swarm

into one automated DevOps workflow.

---

8. Conclusion

This project provides practical implementation of DevOps concepts including source-code management, CI/CD automation, containerization, container registry management, Infrastructure as Code, configuration management and container orchestration.

The most important achievement of the project is that infrastructure creation and application deployment are not performed entirely manually. Jenkins, Terraform and Ansible automate the major stages, while Docker and Docker Swarm provide containerization and orchestration.

The final result is a working web application deployed on a Docker Swarm environment and accessible through the worker node’s public IP address.
